

Introduction

Miao Qiao

The University of Auckland



Database System Topics

- Relational Model, Relational Algebra 代数.
- SQL (Very Important!)
- Storage and Indexing 索引
- ER Model and DB Design
- Relational DB Design and Normalization
- Query Processing
- Query Optimization 优化.
- Transaction

Teaching Team

- Lecturers

- **Miao Qiao (Course Coordinator)**

- Room 303-524, miao.qiao@auckland.ac.nz
 - Office hour: Wednesday 2:30pm - 3:30pm
 - Zoom Meeting ID: <https://auckland.zoom.us/j/3892847086>

- **Gerald Weber**

- Room 303-527, g.weber@auckland.ac.nz
 - Office hour: Fri 2pm.

- Tutors

- Yizhou Dai Email: ydai992@aucklanduni.ac.nz
 - Hongyu Li Email: cli776@aucklanduni.ac.nz
 - Xizhe Zhang Email: xzha593@aucklanduni.ac.nz

Assessment

- Note: Students must obtain a pass in both the Practical (projects + assignments) and Theory (test + exam) work to pass the course as a whole.
- Assignments 35%
 - A1 due Mar 28 11:59 pm 10%
 - A2 due Apr 18 11:59 pm 10%
 - A3 due May 23 11:59 pm 15%
- Projects 15%
 - P1 due May 9 11:59 pm 10%
 - P2 due May 30 11:59 pm 5%
- Late policy:
 - Each assignment/project can be submitted up to 2 days late, with a 10% penalty per day.
- Term test 20% → 只考前六周
- Exam 30% → 前六周 内容不考

Expectations

■ Time Management

- This course is a standard 15 point course and students are expected to spend 10 hours per week on this course.
- Learning Activities: There is a 1-2 hours of preparation required before attending the 1-hour laboratories every week.
- Self-Directed Time: The remaining 4 to 5 hours should be devoted to self-directed research and completing assignments / exam prep.

■ Attendance

- Learning success highly depend on engagement, so please try and attend most lectures and tutorials.
- Please do not come to campus if you are unwell. When you are well again, watch the recordings and view the online content to catchup.

Resources

- Lecture slides, recordings can be found on Canvas
- Reference Textbooks
 - (main) Database System Concepts, 7th edition, by Abraham Silberschatz, Henry F. Korth, S. Sudarshan, Mc Graw Hill.
 - If you want to keep an ebook we have a 20% discount code COMPSCI751 under link <https://www.mheducation.com.au/database-system-concepts-ise-9781260569568-aus>
 - (ref) Designing Data-Intensive Applications, by Martin Kleppmann, O'Reilly Media, Inc. Publisher.

Class Representative

- Select a class rep
- Attends 2 staff student meetings
- Pass on students' feedback to lecturers

Database Applications Examples

- Enterprise
- Manufacturing
- Banking and finance
 - Customer information, accounts, loans, and banking transactions.
 - Credit card transactions
- Universities
 - registration, grades
- Airlines
- Telecommunication
- Web-based services
- Navigation systems

Purpose of Database Systems

- Massive
- Persistent
- Safe
- Multi-user
- Convenience
- Efficiency
- Reliable

data —^{outlining} database. system

University Database Running Example

- In this text we will be using a university database to illustrate all the concepts
- Data consists of information about:
 - Students
 - Instructors
 - Classes
- Application program examples:
 - Add new students, instructors, and courses
 - Register students for courses, and generate class rosters
 - Assign grades to students, compute grade point averages (GPA) and generate transcripts

Data Models

- A collection of tools for describing
 - Data
 - Data relationships
 - Data semantics
 - Data constraints
- Relational model
- Entity-Relationship data model (mainly for database design)
- Graph model (many to many relationships)
- Document (one to many relationships) XML JSON *tree.*
- Key value (one to one relationships)

pair

key

hard to change to other model

Relational Model

domain 范围
snapshot 快照

- Database = a set of relations (names) = a set of tables
- A relation has a set of attributes (names, types/domains)
- A relation has an indefinite set of tuples
- Example of tabular data in the relational model
- Schema & instances

图.模式 实例

Columns / attributes

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows / tuples

dept_name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

(a) The *instructor* table

Relational Model: Attributes

- Domain
- Atomic types / structured types
- NULL**: a special value, indicating that the particular value is unknown.
- The **NULL** value causes complications of many operations
- Example:

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	NULL
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Relational Model: Keys

- Attribute of a relation where every attribute value of the relation is **unique**
- A set of attributes** that are unique: {ID} and {ID, name} are both keys
- Candidate key (minimal) *候选*
- Primary key** *at least 1*
- Why key is important
 - Identify specific tuples
 - Query/index efficiency
 - Reference (**Foreign keys**)

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Relations are Unordered

- Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
- Example: *instructor* relation with unordered tuples

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

機架構圖



Wrap up

- Database application examples
- Purpose of database systems
- Data models
- Relational model
 - NULL values
 - Keys
 - Schema Diagram

FIN

Any questions?

Relational Algebra

关系 代数

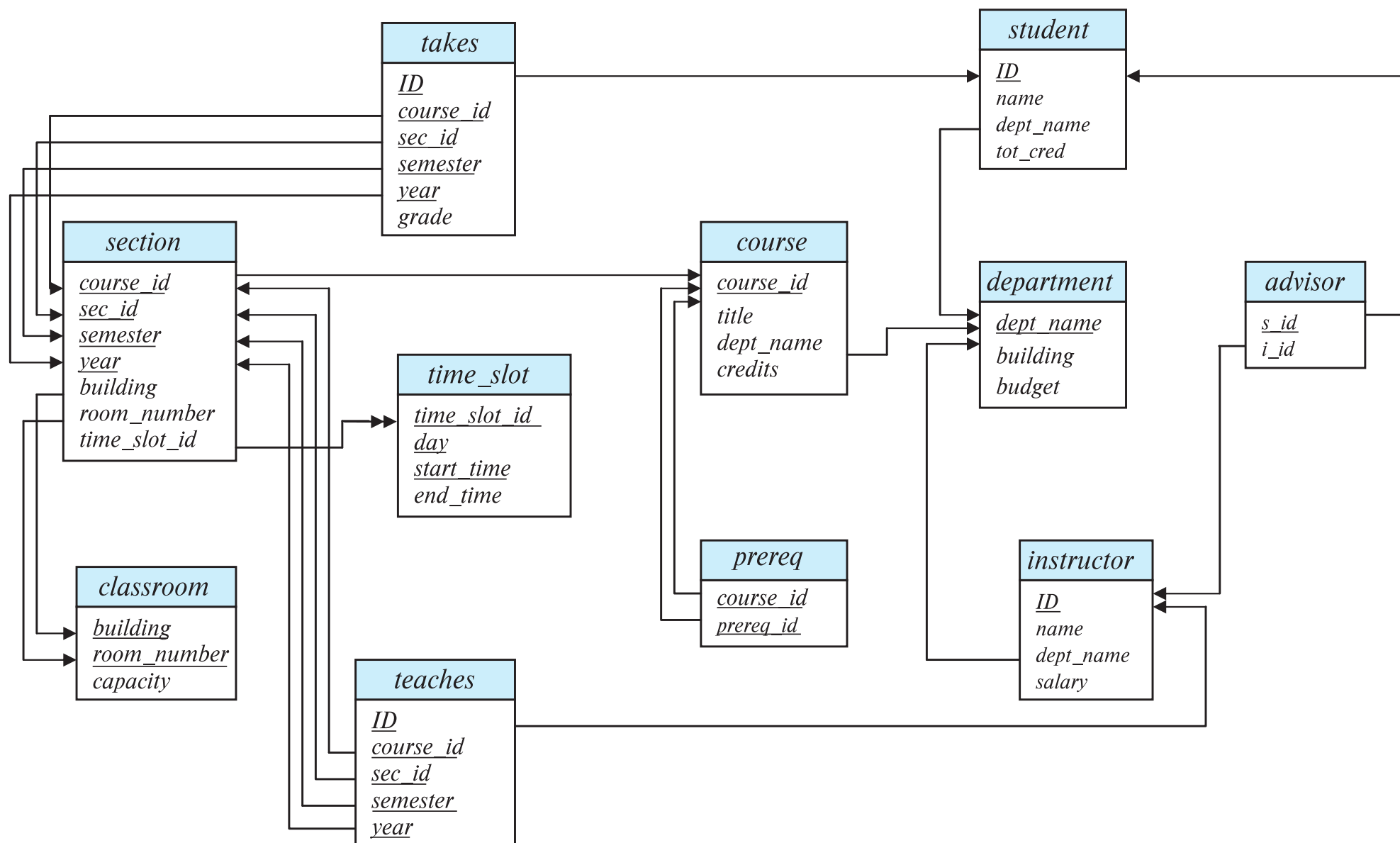
Miao Qiao

The University of Auckland



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

Recap: Schema for University Database



Relational Algebra

- Simplest query: relation name
instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

- Use operators to filter, slice, combine

Select operator: pick certain rows

- Select *instructors* where the instructor is in the “Physics” department.

$\sigma_{dept_name="Physics"}(instructor)$

Select operator

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select row

- Select instructors in Physics depart
with salary > 90000.

$\sigma_{dept_name="Physics" \wedge salary > 90000}(instructor)$

and

- $\sigma_{cond} Rel$
- Comparisons: =, ≠, >, ≥, <, ≤
- Logic connectives:

$\wedge$ (and), $\vee$ (or), $\neg$ (not)

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Project operator: pick certain columns *Columns*

- Pick ID, name and salary of *instructor*

$\Pi_{ID, name, salary} (instructor)$

Select *Σ11*

- Result:

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

relation

$$\Pi_{A_1, A_2, A_3 \dots A_k} (r)$$

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Compose operator: pick both rows and columns

- Find the names of all instructors in the Physics department.

$\sigma_{dept_name = "Physics"}(instructor)$
 $\Pi_{name}(\sigma_{dept_name = "Physics"}(instructor))$

select rows
and columns

- $\Pi_{A1, A2} Expr$
- $\sigma_{cond} Expr$

指定表

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Cross-product: combines two relations (a.k.a Cartesian-product)

instructor X *teaches*

instructor.ID

<i>instructor</i>
<u><i>ID</i></u>
<i>name</i>
<i>dept_name</i>
<i>salary</i>

<i>teaches</i>
<u><i>ID</i></u>
<u><i>course_id</i></u>
<u><i>sec_id</i></u>
<u><i>semester</i></u>
<u><i>year</i></u>

teaches.id

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018

Cross-product: combine two relations (a.k.a Cartesian-product)

instructor X *teaches*

- Find the instructors and the courses that they taught

<i>instructor</i>
<u>ID</u>
name
dept_name
salary

<i>teaches</i>
<u>ID</u>
<u>course_id</u>
<u>sec_id</u>
<u>semester</u>
<u>year</u>

筛选条件，否则是4x5

$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$

Natural join

- Combine two relations, enforce equality on all attributes with same name
- Eliminate the copy of duplicated attributes

instructor	
<u>ID</u> *	
name	
dept_name	
salary	

need common attributes.

instructor ⋈ *teaches*

teaches	
<u>ID</u> *	
course_id	
sec_id	
semester	
year	

- Find all the names of instructors whose department building is 303 and who have taught a course in 2024.

department	
<u>dept_name</u>	
building	
budget	

$$\Pi_{name}(\sigma_{building = '303' \wedge year = 2024}(instructor \bowtie teaches \bowtie department))$$

内部先运行

Cartesian product

Theta join

- $\text{Exp}_1 \bowtie_{\theta} \text{Exp}_2$
 - θ denotes the selection condition
 - Equivalent to $\sigma_{\theta}(\text{Exp}_1 \times \text{Exp}_2)$
- Basic operation implemented in DBMS

缩写.

把选择条件放在了X下面

Wrap up

- Relational algebra
 - Simplest query: relation name
 - Use operators to filter, slice, combine
 - Operators so far: select, project, cross-product, natural join, theta join

Feature	Theta Join (θ -Join)	Natural Join ($\bowtie$)
Condition	Any comparison (e.g., =, >, <, !=)	Only equality (=)
Duplicate Columns?	No removal (keeps all)	Removes duplicates
Symbol	$R \bowtie_{\theta} S$	$R \bowtie S$

Query Quest

a. $\pi_{ID, person_name} (\sigma_{company_name = 'BigBank'} (works))$

employee (ID, person_name, street, city)

works (ID, person_name, company_name, salary)

company (company_name, city)

- Find the Query
 - a. Find the ID and name of each employee who works for “BigBank”.
 - b. Find the ID, name, and city of residence of each employee who works for “BigBank”.
 - c. Find the ID, name, street address, and city of residence of each employee who works for “BigBank” and earns more than \$10000 per year.

b. $\pi_{ID, person_name, city} (\sigma_{company_name = 'BigBank'} (works \bowtie employee))$

c. $\pi_{ID, person_name, street, city} (\sigma_{company_name = 'BigBank' \wedge salary > 10000} (works \bowtie employee))$

> 1000) (employee & company)

Set operation

Union Operation

- Find the names of instructors and the names of the departments

交集

$$\Pi_{name} instructor \cup \Pi_{dept_name} department$$

Intersection operation

- Find names that are both an instructor name and a department name

交集

$$\Pi_{name} instructor \cap \Pi_{dept_name} department$$

<i>instructor</i>	<i>department</i>
<u>ID</u>	<u>dept_name</u>
name	building
dept_name	budget
salary	

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Set difference operation

- Find the names of the departments
who has no instructors

$$\Pi_{dept_name} department - \Pi_{dept_name} instructor$$

去除

The Rename Operation

- Rename operator ρ , name and format the results of an expression
- $\rho_{x(A_1, A_2, \dots, A_n)}(E)$ general form
 - $\rho_x(E)$
 - $\rho_{A_1, A_2, \dots, A_n}(E)$
- Functions:
 - To unify schemas for set operations
 - Find names that are both an instructor name and a department name

$\Pi_{name} instructor \cap \Pi_{dept_name} department$

$\rho_{x(A)}(\Pi_{name} instructor) \cap \rho_{x(A)}(\Pi_{dept_name} department)$

$\swarrow \searrow$
 relation name attributes name

→ 错的, 要保证
名字一样

The Rename Operation

- Rename operator, ρ , name and format the results of an expression

- General form $\rho_{x(A1,A2, \dots An)}(E)$

- $\rho_x(E)$
- $\rho_{A1,A2, \dots An}(E)$

- Functions:

- Unify schemas for set operations
- For disambiguation in "self-joins"
 - Find pairs of instructors in the same dept
 - Instructor x instructor?

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

① $\rho_{r1(a1,b1,c1,d1)} \text{ Instructor } \times \rho_{r2(a2,b2,c2,d2)} \text{ Instructor}$
 $\sigma_{r1.c1 = r2.c2}$

② $\rho_{r1(a1,b1,c,d1)} \text{ Instructor } \bowtie \rho_{r2(a2,b2,c,d2)} \text{ Instructor}$

- Equal Instructor IDs? $a1 \neq a2$
- (A,B) and (B,A)? $a1 < a2$

Assignment Operation

- Break down relational algebra expressions to their parts
- Find all instructor in the “Physics” and Music department.

$Physics \leftarrow \sigma_{dept_name = \text{“Physics”}}(instructor)$

$Music \leftarrow \sigma_{dept_name = \text{“Music”}}(instructor)$

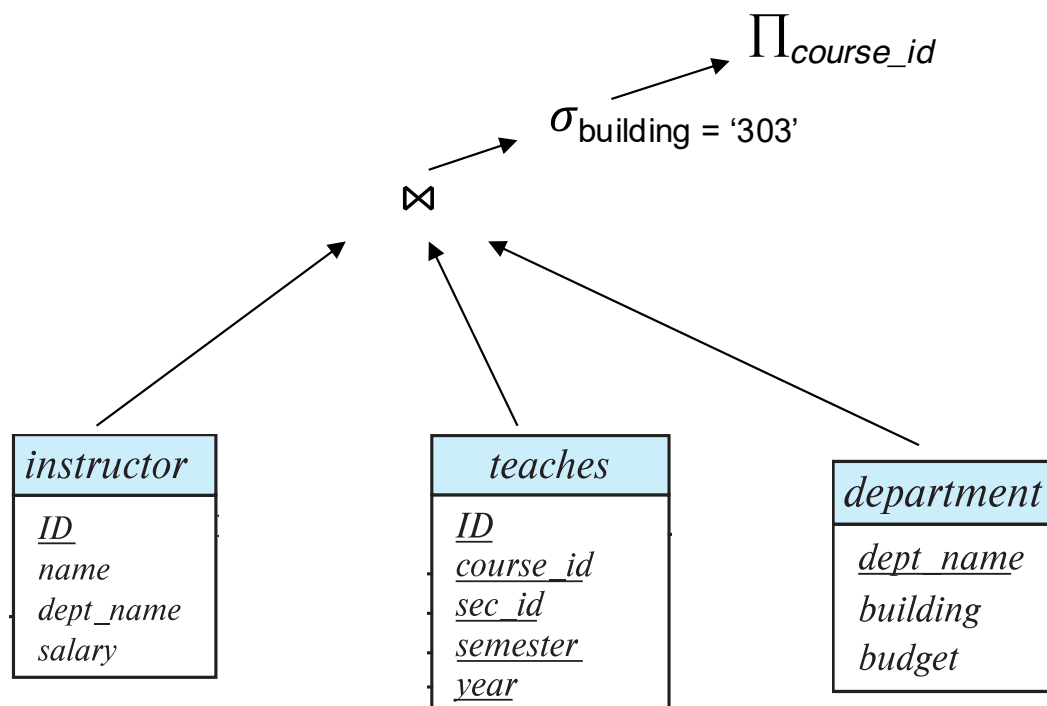
$Physics \cup Music$

easy to eat.

- The assignment operation is denoted by $\leftarrow$ and works like assignment in a programming language.

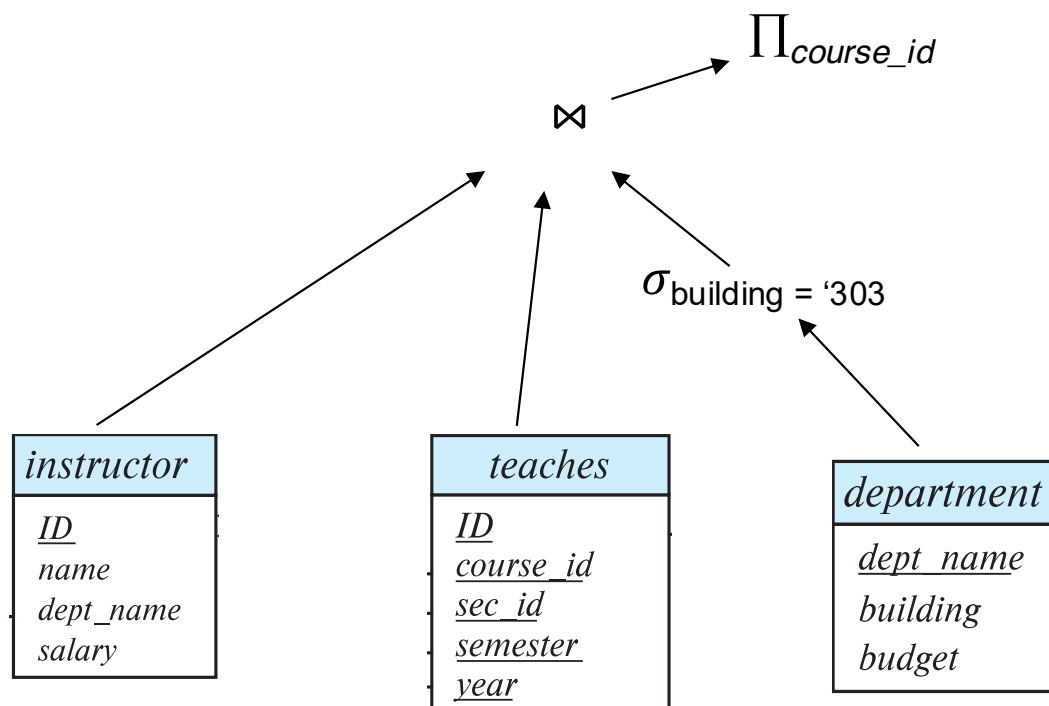
Expression tree

- Find the course IDs whose instructors are in building 303



Expression tree

- Find the course IDs whose instructors are in building 303



- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.

Relational Algebra

- A query language of a set of operations that take one or two relations as input and produce a new relation as their result.
- Operators
 - select: σ
 - project: Π
 - union: $\cup$
 - set difference: $-$
 - Cartesian product: $\times$
 - Natural Join $\bowtie$
 - Theta join $\bowtie_{\theta}$
 - rename: ρ
- Assignment
- Expression tree

Query Quest

$employee (ID, person_name, street, city)$
 $works (ID, person_name, company_name, salary)$
 $company (company_name, city)$

Find the Query

- Find the ID and name of each employee in this database who lives in the same city as the company for which she or he works.

$\pi_{id, person_name} \sigma_{city_employee = city_company}$

$\rho_1(id, person_name, street, city_employee) employee.$

$\rho_2(works) \rho_2(city_company) company$

FIN

Any questions?